FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS – 2º SEMESTRE

JOÃO PEDRO TEIXEIRA DOS SANTOS
DANIEL BANDINI GRANDE
RAPHAEL CARMO MENDES
GUILHERME KOIDE RANGEL

SIMULAÇÃO DE LISTAGEM PAGINADA DE ALUNOS

APLICAÇÃO DE ESTRUTURA DE DADOS NA SOLUÇÃO

Estrutura de Dados – Entrega 2 (Mini-relatório)

SÃO PAULO – SP
2026

1. INTRODUÇÃO

Na Entrega 1 foi desenvolvida a solução **AlunosPaginacao**, uma aplicação de console em C# que simula a listagem paginada de alunos. Os alunos ficam cadastrados em um banco de dados relacional SQLite e, a cada tela, são carregados em uma coleção do C# para serem exibidos.

Este mini-relatório explica como os conhecimentos de estrutura de dados foram aplicados na solução para otimizar a aplicação, apresentando trechos do código que exemplificam cada aplicação.

2. APLICAÇÃO DA ESTRUTURA DE DADOS NA SOLUÇÃO

2.1 Lista dinâmica com apenas os alunos da página

A estrutura principal da solução é a lista `List<Aluno>`, uma lista dinâmica que cresce conforme os elementos são adicionados. A otimização está em **não carregar todos os alunos na lista**: o método `ObterPagina` busca no banco somente os registros da página pedida, usando `LIMIT` (quantidade por página) e `OFFSET` (quantos registros pular), e adiciona à lista apenas esses alunos.

Trecho 1 – Lista preenchida somente com os alunos da página atual

```
int offset = (numeroPagina - 1) * tamanhoPagina;

using var conexao = new SqliteConnection(BancoDados.ConnectionString);
conexao.Open();

using var comando = conexao.CreateCommand();
comando.CommandText = """
    SELECT Id, Nome, Email, Curso, Matricula, DataNascimento
    FROM Alunos
    ORDER BY Id
    LIMIT $tamanho OFFSET $offset;
    """;
comando.Parameters.AddWithValue("$tamanho", tamanhoPagina);
comando.Parameters.AddWithValue("$offset", offset);

var lista = new List<Aluno>();
using var leitor = comando.ExecuteReader();
while (leitor.Read())
{
    lista.Add(new Aluno
    {
        Id = leitor.GetInt32(0),
        Nome = leitor.GetString(1),
        Email = leitor.GetString(2),
        Curso = leitor.GetString(3),
        Matricula = leitor.GetString(4),
        DataNascimento = DateTime.Parse(leitor.GetString(5))
    });
}
```

Fonte: arquivo Data/AlunoRepositorio.cs, linhas 37 a 65, da solução AlunosPaginacao.

Com isso, se a página tem 10 alunos, a lista guarda no máximo 10 objetos, não importa quantos alunos existam no banco. O consumo de memória e o tempo para montar a tela continuam pequenos mesmo que a tabela cresça para milhares de registros. Cada aluno lido é adicionado ao final da lista com o método `Add`, operação rápida, e a lista depois é percorrida com `foreach` para exibir a página.

A consulta também ordena os registros por `Id`, a chave primária da tabela. No SQLite, a tabela já é armazenada em ordem de chave primária, então o banco devolve os alunos ordenados sem precisar reordená-los a cada página, e cada aluno aparece sempre na mesma página.

2.2 Paginação por cálculo de posição

A posição de cada página é calculada por fórmula, sem percorrer elementos. O deslocamento é `offset = (numeroPagina - 1) * tamanhoPagina`, visto no Trecho 1, e o total de páginas é a divisão do total de alunos pelo tamanho da página, arredondada para cima. O método `GarantirPaginaValida` impede que se acesse uma página fora dos limites, assim como se evita acessar uma posição inexistente de um vetor.

Trecho 2 – Cálculo do total de páginas e validação dos limites

```
public static int CalcularTotalPaginas(int totalRegistros, int tamanhoPagina)
{
    if (tamanhoPagina < 1) tamanhoPagina = 1;
    return (int)Math.Ceiling(totalRegistros / (double)tamanhoPagina);
}

public static int GarantirPaginaValida(int paginaDesejada, int totalPaginas)
{
    if (totalPaginas <= 0) return 1;
    if (paginaDesejada < 1) return 1;
    if (paginaDesejada > totalPaginas) return totalPaginas;
    return paginaDesejada;
}
```

Fonte: arquivo Services/PaginacaoService.cs, linhas 8 a 20, da solução AlunosPaginacao.

Com os 57 alunos cadastrados e 10 por página, a aplicação calcula 6 páginas, sendo a última com 7 alunos. Como a posição de qualquer página é obtida diretamente pelo cálculo, as opções “Última página” e “Ir para página” levam à página desejada sem passar pelas anteriores.

2.3 Vetores para a geração dos dados

Para popular o banco na primeira execução, os nomes e os cursos ficam em vetores (`string[]`) fixos. O acesso a um elemento do vetor pelo índice é imediato. Para gerar 57 alunos a partir de 30 nomes, o índice é calculado com o resto da divisão (`i % Nomes.Length`), que volta ao início do vetor quando chega ao fim.

Trecho 3 – Acesso aos vetores de nomes e cursos pelo índice

```
for (int i = (int)totalAtual; i < quantidadeDesejada; i++)
{
    string nomeBase = Nomes[i % Nomes.Length];
    string nome = i < Nomes.Length ? nomeBase : $"{nomeBase} {i / Nomes.Length + 1}";
    string curso = Cursos[random.Next(Cursos.Length)];
```

Fonte: arquivo Data/BancoDados.cs, linhas 86 a 90, da solução AlunosPaginacao.

3. CONCLUSÃO

A solução aplica estrutura de dados para otimizar a listagem de alunos: a lista dinâmica guarda somente os alunos da página exibida, a paginação é feita por cálculo direto de posição, com validação dos limites, a ordenação aproveita a chave primária do banco, e vetores simplificam a geração dos dados de teste. Dessa forma, a navegação entre as páginas continua rápida e com baixo uso de memória, independentemente da quantidade de alunos cadastrados.